Data Science Application Intern - CODTECH

- Presented by Samridhhi Pathak
- Prn : 20230802023
- Mentor : Dr. Sanjay Mohite

Offer Letter - CODTECH IT SOLUTIONS Pvt Ltd.

CIN- U62099TS2023PTC179813

CODTECH IT SOLUTIONS PVT.LTD
INFORMATION TECHNOLOGY SERVICES
CODTECH
8-7-7/2, Plot NO.51, Opp: Naveena School, Hasthinapuram Central, Hyderabad , 500 079. Telangana

Internship Offer Letter

Date : 30/04/2025
Intern ID :CT08DL700

Dear Samriddhi Pathak

Congratulations on being selected for the **Data Science.** We at **CODTECH IT SOLUTIONS PVT.LTD** are thrilled to have you join our team. **This Online Internship will span 8 weeks from May 25th, 2025 to July 25th, 2025.**

This internship is designed as an educational experience, focusing on learning, skill development, and gaining practical knowledge. As an intern, we expect you to:

1. Complete all assignments to the best of your ability.
2. Follow any lawful and reasonable instructions provided by your supervisors.
3. Participate actively in team meetings and discussions.
4. Provide regular updates on your progress.
5. Adhere to company policies and maintain a professional demeanor.
6. Collaborate effectively with team members and contribute to group projects.
7. Seek feedback and apply it to improve your performance.

We trust that you will approach all tasks with diligence and enthusiasm. We are confident that this internship will be an enriching experience for you. We look forward to working with you and supporting you in achieving your career aspirations

Best regards,

Neela Santhosh Kumar
Human Resources & Academic Head
CODTECH IT SOLUTIONS PRIVATE LIMITED

VERIFIED BY
CODTECH IT SOLUTIONS PVT LTD

www.codtechitsolutions.com
+91 9848925128
Hr@codtechitsolutions.com

D Y Patil International University, Akurdi Pune
Established under Maharashtra act No. LXIII of 2017 Sector 29, Pradhikaran, Akurdi Pune (Maharashtra) India 411044
School of Computer Science, Engineering & Applications (SCSEA)

INDUSTRY INTERNSHIP - APPLICATION FORM

Reference #: - DYPIU/SCSEA/Inl/AY 24-25/ A-2023 **Date:** - 2/05/25

Name of Student: Samriddhi Pathak **Batch:** 2023
PRN: 2023080 2023 **Year:** 2nd Year **(IV Sem)**
Form submission date: 2/5/25
Contact No: 7380351817, **E-mail:** 2023080 2023@dypiu.ac.in

To,
Industry Internship In-charges / Coordinator
School of Computer Science Engg. & Applications
D Y Patil International University, Akurdi, Pune-44

Subject: Request to participate in Industry Internship

Respected Sir,
I am a student of School of Computer Science Engg. & Applications from DYPIU studying in 2nd Year (IV semester). I would like to pursue industry internship in below mentioned organization:

Name of the organization: codtech IT Solutions PVT LTD.
Name of the contact person with designation: Neela Santosh Kumar (Human Resource & academic head)
Correspondent address & Phone number of Organization:
8-7-7/2, Plot no. 51, opp: Naveena School, Hasthinapuram Central, Hyderabad, 8374616192
Duration of internship (2.5 Months): - Start date 25 May End date 25 July
I request your permission to attend the above-mentioned internship as a part of my academic curriculum. I will submit the offer letter and completion certificate of the Internship at School of Computer Science Engineering & Applications within given time line.

Thanking you.
Yours sincerely,
Student's signature

1. Industry Internship In-charge, SCSEA
D. Y. Patil International University, Akurdi,Pune

2. Dr. Pragati Choudhari
SY Academic Coordinator
D. Y. Patil International University, Akurdi

3. Overall Internship Coordinator, SCSEA
D. Y. Patil International University, Akurdi, Pune

4. Prof (Dr.) Rahul Sharma
Director SCSEA
D. Y. Patil International University, Akurdi, Pune

Project Overview

- Objective: Data Science Application & Task Fufilment.
- **Tech Stack:**
- Python IDE
- Pandas
- PyTorch
- Scikit Learn

Task 1 : **Data Pipeline Development**

Input & Output

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import joblib


df = pd.read_csv('titanic_sample.csv')

X = df.drop('survived', axis=1)
y = df['survived']

numerical_cols = ['age', 'fare']
categorical_cols = ['sex', 'embarked']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_cols)
    ]
)

pipeline = Pipeline(steps=[
    ('preprocessing', preprocessor),
    ('classifier', RandomForestClassifier(random_state=42))
])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

joblib.dump(pipeline, 'model.pkl')
```

```
PS C:\Users\samri\OneDrive\Desktop\DataPipeline> python -u "c:\Users\samri\OneDrive\Desktop\DataPipeline\main.py"
Accuracy: 1.00
PS C:\Users\samri\OneDrive\Desktop\DataPipeline>
```

Summary Of Task 1 :

1) Data Preparation

- Loaded Titanic dataset and separated features (X) from the target (y).
- Identified numerical (age, fare) and categorical (sex, embarked) columns for preprocessing.

2) Pipeline Construction

- Used ColumnTransformer to scale numerical features and one-hot encode categorical ones.
- Combined preprocessing with a RandomForestClassifier in a Pipeline.

3) Model Training & Evaluation

- Split data into training and testing sets, trained the pipeline, made predictions, evaluated using accuracy_score, and saved the model using joblib.

Task 1 : Conceptual Diagram

DATA PIPELINE DEVELOPMENT

DATA PREPARATION

- Loaded Titanic dataset and separated features (*X*) from the target (y).
- Identified numerical (***age***, ***fare***) and categorical (***sex***, ***embarked***) columns for preprocessing

↓

PIPELINE CONSTRUCTION

- Used ***ColumnTransformer*** to scale numerical features and one-hot encode categorical ones
- Combined preprocesing with a ***RandomForestClassifier*** in a P***Pipeline***

↓

MODEL TRAINING & EVALUATION

- Split data into training and testing sets, trained the pipeline, made predictions, evaluated using `accuracy_score`, nd saved the model using `joblib`

Task 2 : **Image Classification Project**

Input :

```python
import torch
from torch import nn
from torch.utils.data import DataLoader
import numpy as np
from torchvision import datasets
from torchvision.transforms import ToTensor, Lambda, Compose
import matplotlib.pyplot as plt

training_data = datasets.FashionMNIST(
    root = "data",
    train = True,
    download = True,
    transform = ToTensor()
)
test_data = datasets.FashionMNIST(
    root = "data",
    train = False,
    download = True,
    transform = ToTensor()
)

batch_size = 64
train_dataloader = DataLoader(training_data, batch_size = batch_size)
test_dataloader = DataLoader(test_data, batch_size = batch_size)

device = "cuda" if torch.cuda.is_available() else "cpu"

class NeuralNetwork(nn.Module):
    def __init__(self):
        super(NeuralNetwork, self).__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28 * 28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
model = NeuralNetwork().to(device)
print(model)

loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr = 1e-3)

def train(dataloader, model, loss_fn, optimizer):
    size = len(dataloader.dataset)
    model.train()
    for batch, (X, y) in enumerate(dataloader):
        X, y = X.to(device), y.to(device)
```

```python
        pred = model(X)
        loss = loss_fn(pred, y)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if batch % 100 == 0:
            loss, current = loss.item(), batch * len(X)
            print(f"loss: {loss:>7f}  [{current:>5d}/{size:>5d}]")

def test(dataloader, model, loss_fn):
    size = len(dataloader.dataset)
    num_batches = len(dataloader)
    model.eval()
    test_loss, correct = 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)

            pred = model(X)
            test_loss += loss_fn(pred, y).item()
            correct += (pred.argmax(1) == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= size
    print(f"Test Error: \n Accuracy: {(100 * correct):>0.1f}%, Avg loss: {test_loss:>8f} \n")

epochs = 10
for t in range(epochs):
    print(f"Epoch {t+1}\n-------------------------------")
    train(train_dataloader, model, loss_fn, optimizer)
    test(test_dataloader, model, loss_fn)
print("Done!")

torch.save(model.state_dict(), "model.pth")
print("Model saves!")

model = NeuralNetwork()
model.load_state_dict(torch.load("model.pth"))

classes = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]
```

```python
x, y = test_data[0][0], test_data[0][1]
y
with torch.no_grad():
    pred = model(x)
    predicted, actual = classes[pred[0].argmax(0)], classes[y]
    print(f"Predicted: '{predicted}', Actual: '{actual}'")


model.eval()


num_images = 10
plt.figure(figsize=(15, 5))

for i in range(num_images):
    x, y = test_data[i][0], test_data[i][1]
    with torch.no_grad():
        pred = model(x.unsqueeze(0))
        predicted = classes[pred.argmax(1).item()]
        actual = classes[y]

    plt.subplot(2, 5, i + 1)
    plt.imshow(x.squeeze(), cmap="gray")
    plt.title(f"Pred: {predicted}\nActual: {actual}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Output :

```
Test Error:
 Accuracy: 69.4%, Avg loss: 0.819680

Epoch 10
-------------------------------
loss: 0.839777  [    0/60000]
loss: 0.905910  [ 6400/60000]
loss: 0.676214  [12800/60000]
loss: 0.865702  [19200/60000]
loss: 0.762167  [25600/60000]
loss: 0.765854  [32000/60000]
loss: 0.853649  [38400/60000]
loss: 0.818723  [44800/60000]
loss: 0.823679  [51200/60000]
loss: 0.781517  [57600/60000]
Test Error:
loss: 0.823679  [51200/60000]
loss: 0.781517  [57600/60000]
Test Error:
loss: 0.781517  [57600/60000]
Test Error:
Test Error:
 Accuracy: 70.7%, Avg loss: 0.787570

Done!
Model saves!
Predicted: 'Ankle boot', Actual: 'Ankle boot'
PS C:\Users\samri\OneDrive\Desktop\image classification>
```

```
PS C:\Users\samri\OneDrive\Desktop\image classification> python -u "c:\Users\
NeuralNetwork(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear_relu_stack): Sequential(
    (0): Linear(in_features=784, out_features=512, bias=True)
    (1): ReLU()
    (2): Linear(in_features=512, out_features=512, bias=True)
    (3): ReLU()
    (4): Linear(in_features=512, out_features=10, bias=True)
  )
)
Epoch 1
-------------------------------
loss: 2.295467  [    0/60000]
loss: 2.285797  [ 6400/60000]
loss: 2.268692  [12800/60000]
loss: 2.269541  [19200/60000]
loss: 2.245027  [25600/60000]
loss: 2.224855  [32000/60000]
loss: 2.216433  [38400/60000]
loss: 2.191297  [44800/60000]
loss: 2.184205  [51200/60000]
loss: 2.160926  [57600/60000]
Test Error:
 Accuracy: 53.0%, Avg loss: 2.154528
```

Summary Of Task 2 :

1) Data Loading and Preprocessing

- Imported FashionMNIST dataset using torchvision.datasets, and applied ToTensor() transformation.
- Used DataLoader to batch and shuffle training/testing data efficiently.
- Set device to "cuda" if available for GPU acceleration.

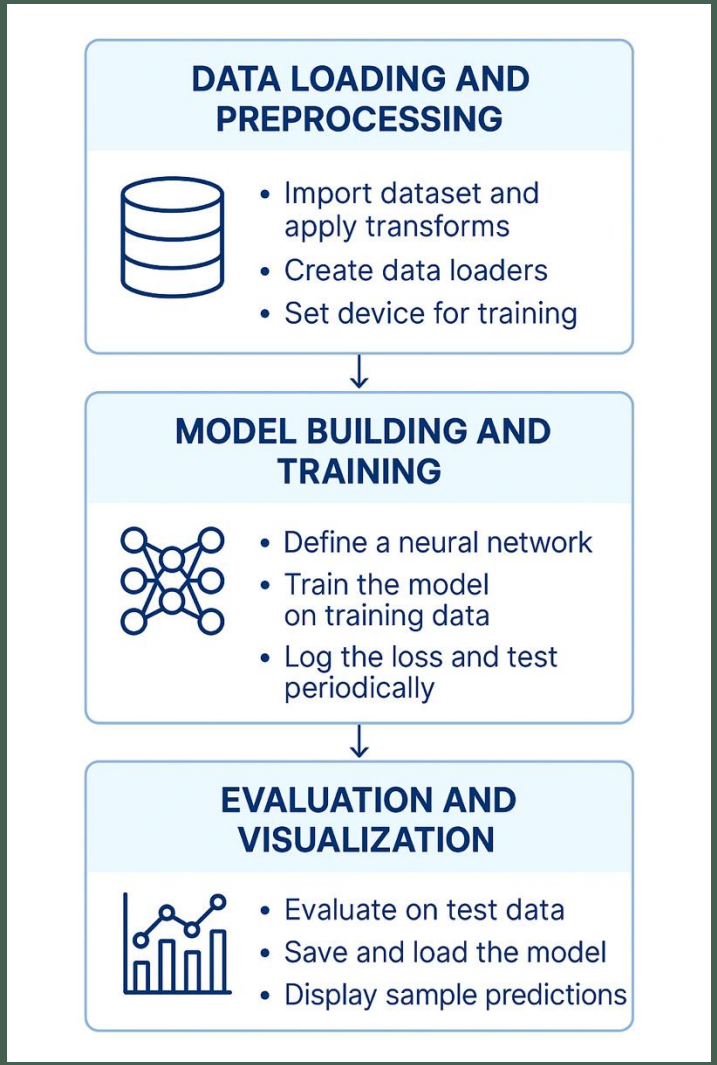
2) Model Building and Training

- Defined a feedforward neural network (NeuralNetwork) with 3 Linear layers and ReLU activations.
- Trained the model using SGD optimizer and CrossEntropyLoss for 10 epochs.
- Logged training loss and periodically evaluated performance on test data.

3) Evaluation and Visualization

- Tested model accuracy and loss using model.eval() and torch.no_grad().
- Saved and reloaded the model with torch.save() and torch.load().
- Displayed predictions for sample images using matplotlib, showing predicted vs actual class labels.

Task 2 : **Conceptual Diagram**



Conclusion

Data Pipeline Development

- A robust data pipeline was created using Pandas and Scikit-learn, streamlining data preprocessing and model training. It ensures modularity, scalability, and easy deployment by integrating transformation and classification into a single workflow.

Image Classification Project

- A complete image classification workflow was built using PyTorch, covering data loading, neural network training, evaluation, and prediction visualization. The modular design supports reproducibility and model reuse across similar tasks.

THANKYOU !